
Explaining Attention with Program Synthesis

Anonymous Author(s)

Affiliation

Address

email

Abstract

A longstanding goal of research on interpretable deep learning is to replace opaque neural computations with human-meaningful symbolic descriptions. In this paper, we propose an approach for approximating the behavior of components of deep networks with executable programs. We focus on attention heads in transformer language models. For a given head, we first compute its associated attention matrices on a collection of randomly selected training examples. Next, we prompt a pre-trained language model with a summary of these matrices, and instruct it to generate a set of Python programs that can reproduce the associated attention patterns given only text from the input sentence. Finally, we re-rank programs according to how well our final set of programs predict behavior on held-out inputs. We demonstrate that a set of fewer than 1,000 such generated programs can reproduce the attention patterns of heads in GPT-2, TinyLlama-1.1B, and Llama-3B, achieving an [bzl: average?] Intersection-over-Union similarity above 75% across three datasets spanning natural language, code, and mathematics. Moreover, the best-fit programs can *replace* neural attention heads without substantially affecting model behavior: replacing 25% of attention heads with programmatic surrogates across the three models incurs only a 16% average perplexity increase, while maintaining performance on a variety of downstream question answering benchmarks. This work contributes a scalable pipeline for reverse-engineering attention heads in transformer models using human-readable, executable code, advancing a path toward symbolic transparency in neural models.

1 Introduction

Understanding the computations performed by deep networks in algorithmic terms is a long-standing problem in machine learning [13, 37, 23]. Past work approaching this problem has attempted to assign meaning to neurons or other distributed features in a top-down way (by training probes for human-defined concepts of interest [17, 29]), or in a bottom-up way (by generating summaries of the inputs that activate a feature, or the outputs that the feature promotes [22, 16, 2]). While powerful, these methods stop short of providing a full, formal description of neural computation. Current work addresses this by labeling intermediate components of deep networks with natural language descriptions [4, 8], whose interpretations may themselves be hard to formalize or open to debate.

In this paper, we develop an alternative approach to deep network interpretability based on program synthesis. To explain the computation performed by some component of a deep network, we search for a piece of executable code that approximates the computation performed by that component. Executable programs occupy a natural middle ground between the complexity of billion-parameter models and natural language explanations; they offer a medium that is both human-readable and formally verifiable [2]. Unlike natural language descriptions, programs can be directly substituted for neural components, enabling causal validation of inferred explanations and opening a path toward model editing grounded in interpretable logic rather than weight manipulation.

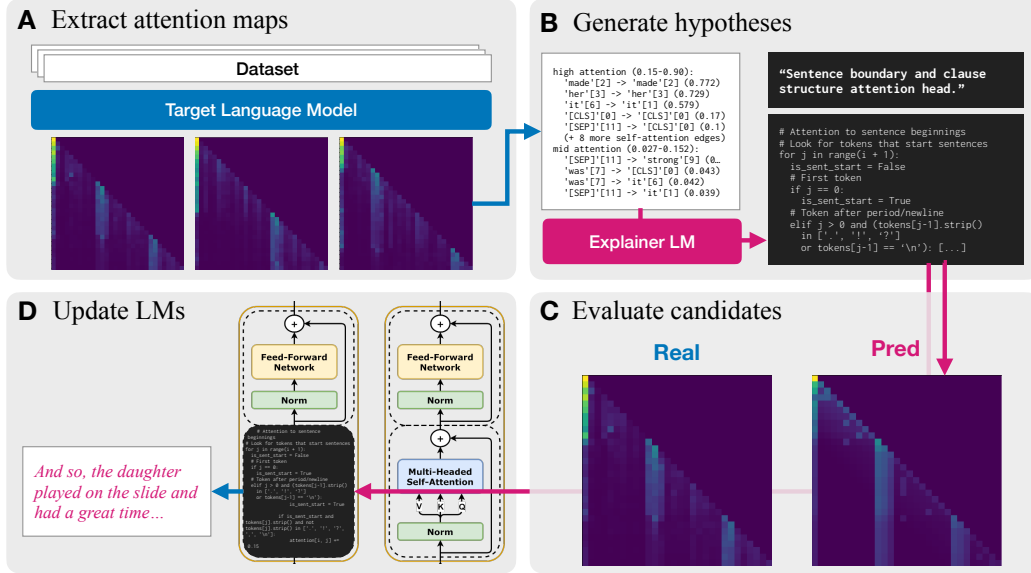


Figure 1: Synthesizing programmatic representations of attention heads in transformer models. Clockwise from top left: **A** Given a target model for explanation, we extract its attention pattern on a collection of representative inputs. **B** We summarize these attention patterns in a textual prompt, and instruct an explainer language model to synthesize one or more candidate Python programs that can reproduce attention patterns given only input text. **C** Candidate programs are compared to original attention patterns, and the highest-scoring ones are selected. **D** These programs can be directly inserted into models, replacing learned attention heads with symbolic programs.

We focus on generating programmatic explanations of attention heads in transformer language models (LMs) [30]. For each attention head in a model of interest, we extract a set of example attention maps on training examples, then prompt another LM to generate a set of candidate Python programs that can reproduce those attention maps given only input text (Figure 1). Finally, we re-rank this collection of programs for each head in a model to obtain a best-fit program for each attention head. Across BERT-Base, GPT-2-Small, TinyLlama-1.1B and Llama-3B language models, we find that a substantial fraction of attention heads can be approximated with executable programs to a high degree of accuracy. We test whether the programs we find recover their associated heads’ casual function by replacing heads with programs. We find that as many as 25% of heads can be replaced while incurring only a 16% increase in perplexity, and without a substantial effect on downstream question answering performance on several benchmarks.

Our results show that, even in state-of-the-art language models, a substantial fraction of attention patterns can be understood in symbolic terms; so much so that we can actually replace trained neural components with symbolic surrogates via executable code without substantially changing model behavior. More generally, they highlight the utility of modern (LM-driven) program synthesis methods as an alternative framework for approaching the broader question of how deep networks operate.

2 Background

2.1 Interpreting Deep Networks

There are several levels at which one can attempt to interpret transformer-based neural language models. At the most granular level, individual neurons can be examined to determine what information they encode [29]. This approach can provide granular insight but can suffer from a lack of generalizability [8]. Conversely, model-level analyses consider how layers collectively contribute to downstream predictions, revealing high-level strategies while obscuring the contributions of specific circuits [4]. At the attention head level, one can analyze how heads distribute focus across input sequences [31]. This approach balances interpretability and simplicity, as attention heads often track

a variety of linguistic patterns, many of which can be assigned definitive roles. Next, model level analyses consider how all components collectively contribute to downstream predictions; this reveals high-level strategies but obscure contributions of more granular elements such as attention heads.

2.2 Explaining Attention

Attention weights determine how much the contents of each token in a prior layer map onto the contents of each token in the next layer. Prior work has examined these attention weights as a form of *attribution map* – by looking at where the model is attending to at each layer, we get a rough sense of which tokens are most important to a model’s prediction [18]. By looking at these attention weights across layers, we also get a sense of where and how information is being directed within the model [31], which can be used as a signal for what circuits and internal mechanisms that a model might be using [12]. Understanding attention provides insight into a key component of the computational process underlying model predictions, offering a stepping stone toward their explanation.

Closer examination of attention shows that it frequently reflects structured, functionally specialized behavior within models. Certain attention heads in BERT consistently focus on specific syntactic relationships, such as subject-verb agreement and direct objects [6]. While some heads exhibit specialized, interpretable roles, others remain diffuse or task-agnostic. Specific heads are also critical for integrating information during in-context learning; removing these components impairs model adaptation, indicating that particular heads play distinct, necessary roles in combining and propagating for task-relevant information [36].

2.3 Interpretability and Program Synthesis

Closely related to the present work, [22] approximate the behavior of neurons in vision models and small transformer encoders by performing enumerative search for simple logical expressions that reproduce their behavior. A parallel line of work in automated interpretability uses language models to generate natural language descriptions of neurons and features [2], but such descriptions lack formal verifiability and cannot be directly substituted into model computations. A separate line of work develops transformers that are interpretable by design, either by compiling human-written RASP programs into transformer weights [34] or by training modified transformers that can be automatically converted into discrete, human-readable programs [14]; these approaches characterize transformer computation formally but require architectural constraints or training from scratch rather than post-hoc analysis of pretrained models. [20] develop MIPS, which converts RNNs trained on algorithmic tasks into finite state machines and applies symbolic regression to distill their behavior into Python code; they explicitly identify generalization to transformer architectures and scaling to larger networks as open directions for future work.

Our method can be understood as a natural extension of this program: we apply LM-guided program synthesis directly to attention heads in modern autoregressive language models, targeting the richer space of Python programs with linguistic and structural logic rather than compact boolean or integer expressions. Where past work was limited to simpler expressions over small or specialized architectures, we scale program synthesis to GPT-2, TinyLlama-1.1B, and Llama-3B, and crucially validate our approximations causally by inserting synthesized programs into live model forward passes. In doing so we build on a long line of recent work on LM-guided program synthesis [1, 5, 24]. As a result, our method can produce what we believe to be the first programmatic explanations that scale to modern LMs, and the first evidence that these programs can actually be inserted into LMs while preserving their capabilities.

3 Approach

3.1 Method Overview

We seek to approximate the internal logic of a transformer model $\mathcal{M}$, which contains (among other modules) a set of attention heads $\mathcal{A}_i$. For a given input sequence of tokens X , each head $\mathcal{A}_i$ within $\mathcal{M}$ projects hidden representations H into queries (Q) and keys (K) to produce an **attention matrix**:

$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) \quad (1)$$

where $A \in \mathbb{R}^{n \times n}$ represents the directed weights between all token pairs in a sequence of length n . These attention weights then direct how much each token in a previous layer affect the token position of the current layer. Given $\mathcal{M}$, we wish to find for each $\mathcal{A}_i$ some symbolic program π that can map directly from $X \mapsto A$. Our framework, illustrated in Figure 1, proceeds in four steps:

Attention Map Extraction We first record the ground-truth activations of $\mathcal{L}$ across a corpus of sequences processed by the target model $\mathcal{M}$. This provides a set of empirical attention matrices $\{A_1, A_2, \dots, A_n\}$ that serve as the targets for our programmatic approximations.

Program Synthesis We define a space of symbolic programs Π , where each $\pi \in \Pi$ is an executable Python function that takes X as input and outputs a hypothesized attention matrix $\hat{A} = \pi(X)$. We utilize an interactive program synthesis agent $\mathcal{S}$ to generate these programs based on the patterns observed in the extracted maps; the specific synthesis strategies are described in Section 3.2.

Alignment Scoring To quantify the fidelity of a proxy, we compute the distance $\Delta(A, \hat{A})$ between the neural and symbolic matrices. During program synthesis and refinement, we rank candidate programs using Jensen-Shannon distance (JSD). For all downstream evaluation and causal validation experiments, we adopt *Intersection over Union* (IoU), which offers significant reductions in computational overhead and assigns a pair of attention maps a similarity score $\delta \in [0, 1]$.

$$\text{IoU}(A, \hat{A}) = \frac{\sum_{i,j} \min(A_{i,j}, \hat{A}_{i,j})}{\sum_{i,j} \max(A_{i,j}, \hat{A}_{i,j})} \quad (2)$$

Causal Validation Finally, we perform interchange interventions [15] by replacing attention heads with their programmatic surrogates. For a specific downstream task, we replace the neural attention matrix A with the programmatic output $\pi(X)$ during the model’s forward pass. Given the computational cost of causal intervention, we use the IoU scores from the previous as a filtering proxy, only attempting replacement for heads where high-fidelity symbolic proxies have been identified. By measuring the resulting change in perplexity and task accuracy, we validate whether the proxy captures the necessary attention-level features to maintain the model’s functional performance.

3.2 Generating Functional Proxies via Program Synthesis

To generate the symbolic program space Π , we utilize an auxiliary large language model as a synthesis agent, denoted $\mathcal{S}$. Given a target attention matrix A and the corresponding input sequence X , $\mathcal{S}$ is prompted to produce an executable Python function π such that $\pi(X) \approx A$. Candidates are validated for syntax and executability, then scored against real attention using mean similarity, with failed executions assigned maximal error. For each head, a single feedback-conditioned refinement step follows: we identify representative best and worst-scoring examples, construct structured error feedback by contrasting real and predicted attention patterns, and prompt $\mathcal{S}$ to produce one revised program π' . The refined program is re-validated and re-scored under the same similarity objective. The optimal proxy π^* is then selected by maximizing IoU alignment over held-out data sequences $(X', A') \in \mathcal{D}_{\text{val}}$.

Concretely, our pipeline first extracts attention activations, filtering for the top 2.5% of activations by magnitude to isolate the most salient head behaviors. These filtered activations are formatted as token-pair weight summaries and embedded into structured prompts of approximately 4,000 tokens. The synthesis agent $\mathcal{S}$ is provided access to NumPy, spaCy and NLTK for numerical and linguistic processing. The full program library used across all four models and used for all results in this paper was produced from fewer than 4,000 candidates using Claude Sonnet 4 at approximately \$150 in total API cost (roughly 35 million input tokens and 3.5 million output tokens). The final set of programs is composed of one program for each head in the four models, or 1,664 programs total.

3.3 Evaluation Details

Datasets We analyze activations across three distinct datasets to ensure a broad coverage of linguistic, mathematical, and logical patterns: TinyStories [11], tiny-codes [25], and Math500 [19]. These datasets were selected for their relative simplicity, which provides an advantage for mechanistic interpretability: using simple, structured data simplifies the act of isolating specific

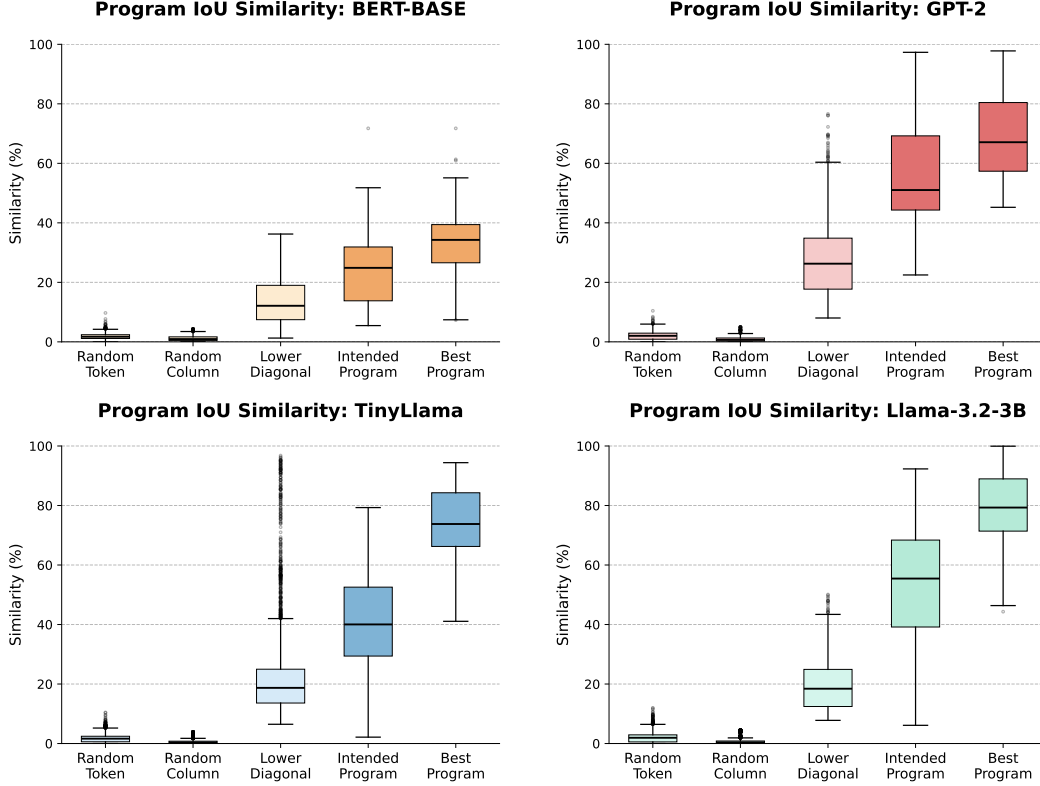


Figure 2: Analysis of program Intersection-over-Union similarity scores across all model attention heads. In general, heads in autoregressive models are easier to fit than in bidirectional models; fit quality increases with model scale. In many cases, the program synthesized for a given head (Intended Program) is outperformed by a program synthesized for a distinct head (Best Program).

160 head behaviors without the stochastic noise inherent in complex corpora [32]. No preprocessing or
 161 sequence truncation was performed on these datasets. Maintaining full sequence lengths allowed
 162 for a more natural observation of activation patterns as they evolve over a long-range context. The
 163 diversity of natural language, code, and mathematics ensures that the resulting programs represent a
 164 breadth of generalized behaviors rather than domain-specific patterns.

165 To evaluate model performance following programmatic head replacement, we utilize an evaluation
 166 suite comprising six benchmark datasets: HellaSwag [38], PIQA [3], SciQ [35], ARC-Easy [7],
 167 Social IQA [28], and COPA [27, 33]. We sample 200 questions from each benchmark. This suite
 168 provides a robust measure of the model’s remaining functional capacity across multiple domains,
 169 including commonsense reasoning, scientific knowledge, and causal inference and was selected
 170 such that all included base language models could perform well enough that our head replacement
 171 interventions reflect the model’s ability to reason as new programs are introduced.

172 **Models** We perform our analysis on four transformer architectures: BERT-base [9] containing
 173 144 heads, GPT-2-small [26] containing 144 heads, TinyLlama-1.1B [39] containing 704 heads, and
 174 Llama-3B [10] containing 672 heads across 28 layers.

175 4 Experiments

176 4.1 Model-Level Analysis

177 To evaluate the general similarity of our synthesized programs to the behavior of each model’s
 178 attention heads, we perform a comparative analysis summarized in Figure 2. We include two random
 179 baselines: Random Token, which assigns each query’s full attention to a randomly selected token

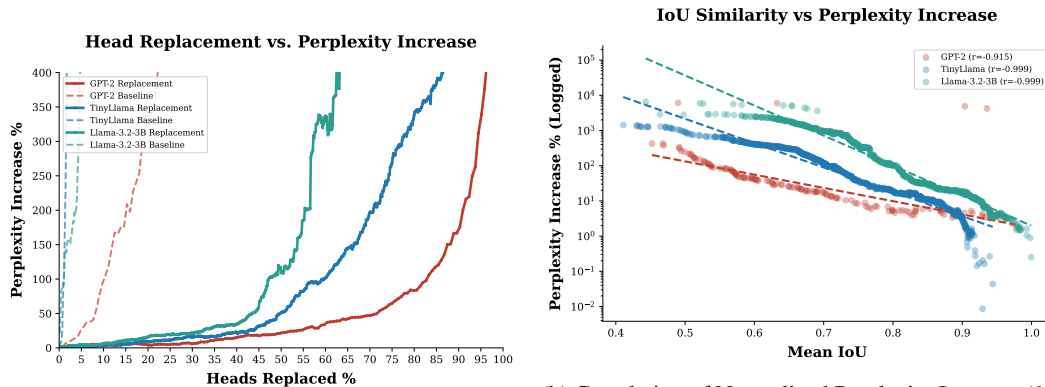
in each row, and Random Column, which concentrates attention uniformly on a single column. We additionally include a “uniform attention” baseline that assigns attention fully to the lower diagonal. We compare these against two program-based conditions: the *intended* program synthesized specifically for each head, and the *best* program, defined as the highest-scoring selected by maximizing IoU across the library of all programs synthesized for all heads in a model. (The latter procedure is related to quality–diversity optimization algorithms like MAP-Elites [21].) [bzl: I still think a entirely first-token attention baseline would be useful here (since that’s the predominant class)]

We find that the best program for each head significantly outperforms all baselines across every model. The best program also consistently exceeds the intended program, reflecting that some synthesized functions are sufficiently general to approximate multiple heads with high fidelity. We further observe a systematic difference between encoder and decoder models, and a positive relationship between model scale and IoU similarity. BERT is comparatively poorly characterized by its synthesized programs relative to its decoder counterparts, which we attribute to the masked language modeling objective producing bidirectional attention distributions that are more complex and less amenable to symbolic approximation. Decoder models, by contrast, are constrained to attend only to preceding tokens, reducing the search space and making individual head behaviors more stereotyped. Notably, IoU scores also increase with model scale: GPT-2 achieves a mean [bzl: should this be median instead? IIRC box plots usually show medians?] best-program IoU of 69%, TinyLlama-1.1B of 74%, and Llama-3B of 79%. We hypothesize this reflects functional specialization at scale, where a larger number of attention heads leads each individual head to serve a narrower and more symbolically tractable role.

4.2 Best-Fit Head Analysis

Figure 4a and 4b provide a granular, head-level visualization of program alignment across GPT-2, pairing an IoU score heatmap (a) with a program category heatmap (b) in which each head is colored by the functional category of its best-fit program. From (a), we observe high alignment across the majority of heads, with later layers appearing to contain a greater number of heads that are easily explained by our set of programs. From (b), we see that program categories are not uniformly distributed across depth: first-token programs dominate early layers, while syntactic programs account for a larger share of middle-layer assignments. The six categories were developed post-hoc by prompting the synthesis agent to group the full program library by functional hypothesis, providing a coarse but interpretable decomposition of the symbolic logic present in the library.

4.3 Head Replacement Analysis



(a) Normalized Perplexity Increase (%) versus the quantity of attention heads replaced by programs.

(b) Correlation of Normalized Perplexity Increase (%) to IoU Similarity in Head and Program pairs [bzl: what are the ones that go under 10^0 :O]

Figure 5: Perplexity remains low when high-IoU heads are replaced first (left), consistent with the strong negative correlation between programmatic alignment and substitution cost across head-program pairs (right).

[illegible]

Figure 1 displays two heatmaps illustrating attention weights for the sentence "Can you send me the report on the new product?". The left heatmap shows attention weights for the original sentence, and the right heatmap shows attention weights for the sentence with tokens replaced by their corresponding word embeddings. The heatmaps are color-coded from dark blue (low attention) to yellow (high attention). The right heatmap includes a legend for the attention weights, ranging from 0.00 to 0.15, with labels for "Strong attention to structural tokens", "Moderate attention to first token", "Self-attention baseline", "Small recency bias for recent tokens", and "Boost for tokens that are whitespace or empty".

Figure 1 displays two heatmaps showing attention weights for the sentence "The cat sat on the mat and the dog ran in the yard." The top heatmap, labeled "[CLS]", shows attention weights for the [CLS] token. The bottom heatmap, labeled "[SEP]", shows attention weights for the [SEP] token. The x-axis and y-axis for both heatmaps are labeled with tokens: [CLS], [CLS], together, the, cat, sat, on, the, mat, and, the, dog, ran, in, the, yard, [SEP]. The color scale ranges from 0.0 (dark purple) to 1.0 (yellow). The [CLS] token attends most strongly to the [CLS] token and the [SEP] token. The [SEP] token attends most strongly to the [CLS] token and the [SEP] token.

[CLS]

	[CLS]	[CLS]	together	the	cat	sat	on	the	mat	and	the	dog	ran	in	the	yard	[SEP]
[CLS]	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
[CLS]	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
together	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
the	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
cat	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
sat	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
on	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
the	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
mat	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
and	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
the	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
dog	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0
ran	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
in	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0
the	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
yard	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
[SEP]	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0

[SEP]

	[CLS]	[CLS]	together	the	cat	sat	on	the	mat	and	the	dog	ran	in	the	yard	[SEP]
[CLS]	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
[CLS]	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
together	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
the	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
cat	0.0	0.0	0.0	0.0													

To move beyond similarity metrics and explore causal evidence of functional alignment, we perform a series of interchange interventions where original attention activations are completely replaced by the outputs of our symbolic proxies. We use downstream perplexity as our first evaluative proxy because it serves as an objective measure of the model’s linguistic coherence; as we replace internal logic with results from external code, the resulting change in perplexity indicates how successfully the programs mimic the head’s task-relevant logic. Our experimental setup tracks the Normalized Perplexity Increase (%) on the y-axis against the percentage of heads replaced on the x-axis. Normalization is employed as perplexity values derived from different datasets and model states exist in distinct distributions; by calculating only the increase from the unedited model, we isolate the impact of the intervention itself. For reference, we compare to a positional baseline in which all heads are replaced with a lower diagonal matrix.

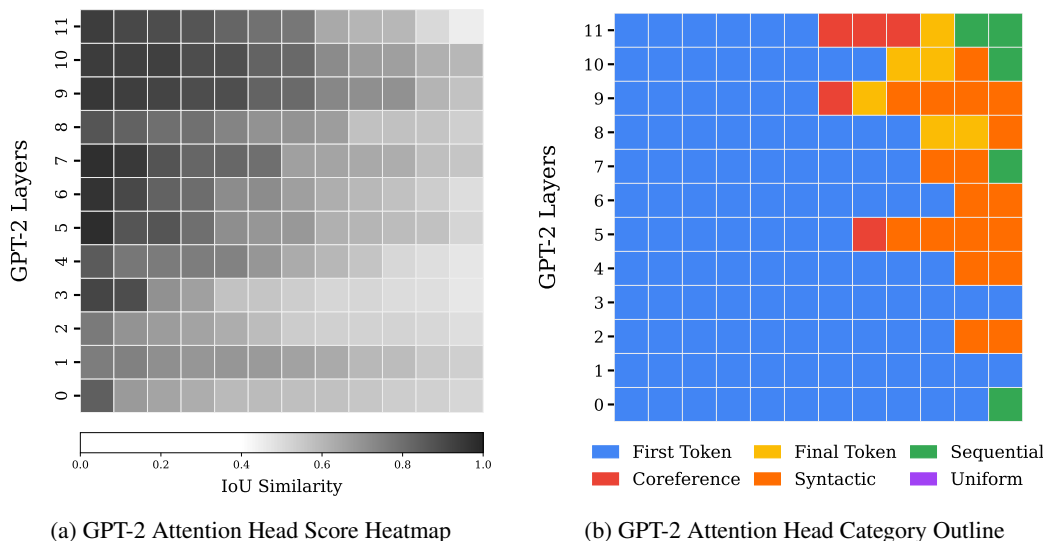


Figure 4: GPT-2 similarities and program types by layer. (a) Attention head accuracies (darker is more accurate). We sort the heads in each layer, and observe that earlier layers are generally harder to approximate than later ones. (b) Attention head types, automatically clustered based on program docstrings. Here we sort each row by type (so note that a given cell of (a) does not necessarily correspond to the same cell of (b)!). A substantial fraction of heads in GPT-2 are well approximated by some form of first-token attention (individual implementations may behave slightly differently on other tokens), with linguistic and discourse functions appearing deeper in the model (q.v. [29]).

Figure 5a illustrates the normalized perplexity increase across models when original attention activations are replaced by either the positional baseline or our synthesized programs. While the structural baseline triggers sharp exponential degradation in perplexity exceeding 1000% after only 5% replacements in Tiny-Llama, our executable programs maintain a low and steadily increasing perplexity. We further observe that the composition of the replacement set significantly affects performance, as shown in Figure 5a. By utilizing IoU scores to guide selection of replacement sets, we can identify orderings that minimize perplexity increases, which is essential for the stable evaluations reported in our primary results. Figure 5b further validates this relationship quantitatively: across all head-program pairs, programmatic alignment and downstream perplexity increase exhibit absolute Spearman correlations of above 0.9 for all models, indicating that IoU score is a reliable predictor of substitution cost.

Figure 6 summarizes the performance of models across varying degrees of programmatic substitution, targeting ten replacement thresholds. Heads are replaced greedily in descending order of IoU score, representing a replacement trajectory in which the best-characterized heads are substituted first. These results indicate that the programmatic approximations maintain high functional utility, particularly up to the 40% replacement threshold for all models. Even at higher replacement levels, where the models are increasingly driven by synthesized code, the performance on several benchmarks does not collapse until 100% replacement. These results demonstrate that programmatic substitution of up to 30–40% of attention heads does not significantly degrade downstream task performance, a finding that holds across all four architectures tested.

5 Discussion

The results presented in this work demonstrate that a symbolic approach to mechanistic interpretability is not only feasible but could yield direct proxies for complex neural activations. Model-level analyses in Figure 2 show that executable programs can achieve up to 99% mean IoU similarity against observed attention activations on a set of sentences with only one refinement of the initial program. Our evaluation in Figure 5 also demonstrates that there is a clear negative correlation between the mean IoU similarity of a program and head and the perplexity increase as a result of replacing the head with its program. Coupled with the replacement model’s performance trends on downstream

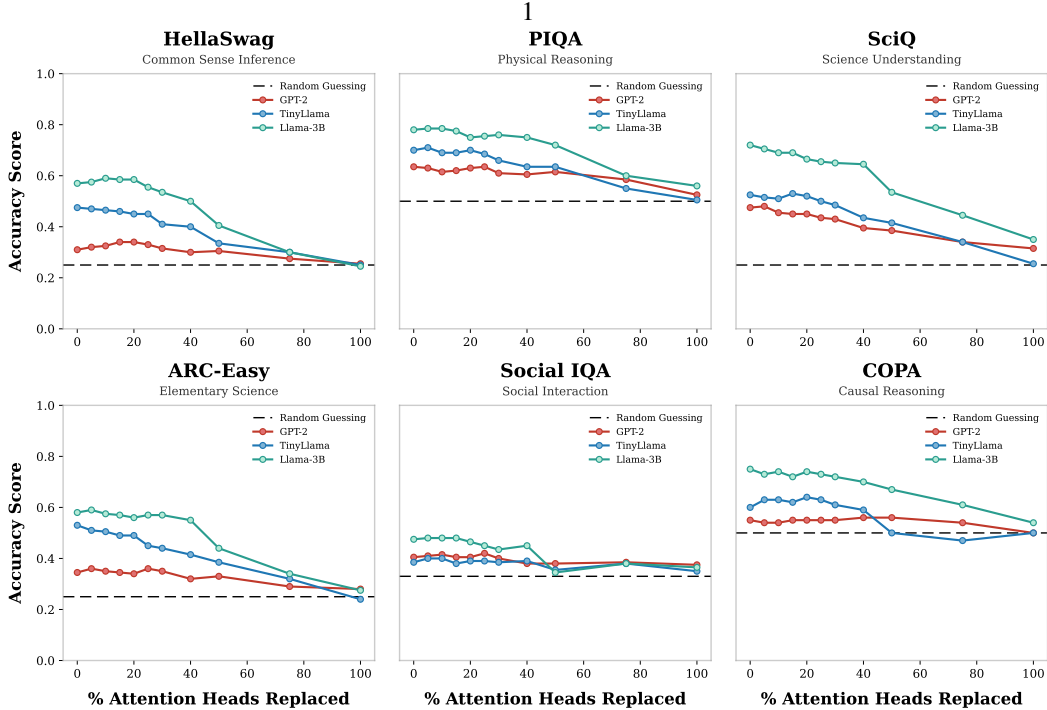


Figure 6: Effect of replacing attention heads on downstream model evaluations.

question-answering evaluations, this seems to suggest that finding the highest IoU similarity program π for each head would allow us to completely replace attention heads with programs without losing ability. This ability could provide a completely new avenue for understanding model internals.

Limitations There is significant room for improvement in expanding the diversity and complexity of these hypotheses. Currently, no model’s attention heads are fully characterized: a substantial fraction of heads achieve IoU scores below 40%, and we attribute the question-answering degradation observed at high replacement levels in Figure 6 primarily to these poorly fit heads. We further observe that many high-scoring programs are not particularly complex, and that the improvement in question-answering seen for some models at low replacement levels may be akin to a normalization effect. Closing this gap likely requires both richer synthesis strategies such as multi-round refinement with stronger feedback signals, and extension of the method beyond output activations to query and key projections, which carry complementary information about attention routing logic. The goal of this paper is to establish that a small, curated program library can serve as faithful proxies for real attention heads with few refinements, providing a concrete foundation for future work toward complete symbolic characterization of transformer architectures.

Next steps A primary objective for subsequent research is to achieve a complete characterization of smaller models targeting 100% head similarity across all heads in fundamental models such as BERT and GPT-2. By bridging the gap between neural activations and symbolic code, we aim to demonstrate that complex model behaviors can be distilled into human-readable logic, allowing us to completely trace the allocation of attention throughout the entire architecture. Ultimately, a complete symbolic characterization of attention would allow researchers to reason about model behavior the way they reason about algorithms: by reading, modifying, and testing the underlying logic directly.

Impact Statement

We do not anticipate any harms or misuses associated with the methods described in this paper.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [2] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu, and William Saunders. Language models can explain neurons in language models. *OpenAI Blog*, 2023.
- [3] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- [4] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Cammarata, Catherine Olsson, Christopher Olah, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [6] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does bert look at? an analysis of bert’s attention. *arXiv preprint*, June 2019. arXiv:1906.04341.
- [7] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint*, 2018. arXiv:1803.05457.
- [8] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint*, 2023. arXiv:2309.08600.
- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019.
- [10] Abhimanyu Dubey, Akhil Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [11] Ronen Eldan and Yuanzhi Li. Tinstories: How small can language models be and still speak coherent english? *arXiv preprint*, 2023. arXiv:2305.07759.
- [12] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Schiefer, Tristan Hume, Josh S. Lefkowitz, Christopher Olah, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.
- [13] Dumitru Erhan, Yoshua Bengio, Aaron Courville, and Pascal Vincent. Visualizing higher-layer features of a deep network. Technical Report 1341, University of Montreal, 2009.
- [14] Dan Friedman, Alexander Wettig, and Danqi Chen. Learning transformer programs. *arXiv preprint arXiv:2306.01128*, 2023.
- [15] Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstraction for the interpretability of deep learning models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [16] Evan Hernandez, Sarah Schwettmann, David Bau, Teona Bagashvili, Antonio Torralba, and Jacob Andreas. Natural language descriptions of deep visual features. *International Conference on Learning Representations (ICLR)*, 2022. arXiv preprint.

- [17] John Hewitt and Christopher D. Manning. A structural probe for finding syntax in word representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.
- [18] Sarthak Jain and Byron C. Wallace. Attention is not explanation. *arXiv preprint*, May 2019. arXiv:1902.10186.
- [19] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2023.
- [20] Eric J. Michaud, Isaac Liao, Vedang Lad, Ziming Liu, Anish Mudide, Caden Juang, Nikolay Bultakov, and Max Tegmark. Opening the AI black box: Program synthesis via mechanistic interpretability. *arXiv preprint arXiv:2402.05110*, 2024.
- [21] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [22] Jesse Mu and Jacob Andreas. Compositional explanations of neurons. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [23] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. *arXiv preprint*, 2023. arXiv:2304.14997.
- [24] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? *arXiv preprint arXiv:2306.09896*, 2023.
- [25] Nam Pham. Tiny codes: A synthetic dataset for reasoning with language and code. <https://huggingface.co/datasets/nampdn-ai/tiny-codes>, 2023.
- [26] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.
- [27] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *2011 AAAI Spring Symposium Series*, 2011.
- [28] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan LeBras, and Yejin Choi. Social IQa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [29] Ian Tenney, Dipanjan Das, and Ellie Pavlick. Bert rediscovers the classical nlp pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
- [30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [31] Jesse Vig. A multiscale visualization of attention in the transformer model. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 2019.
- [32] Elena Voita, Rico Sennrich, and Ivan Titov. The bottom-up evolution of representations in the transformer: A study with machine translation and language modeling objectives. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [33] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

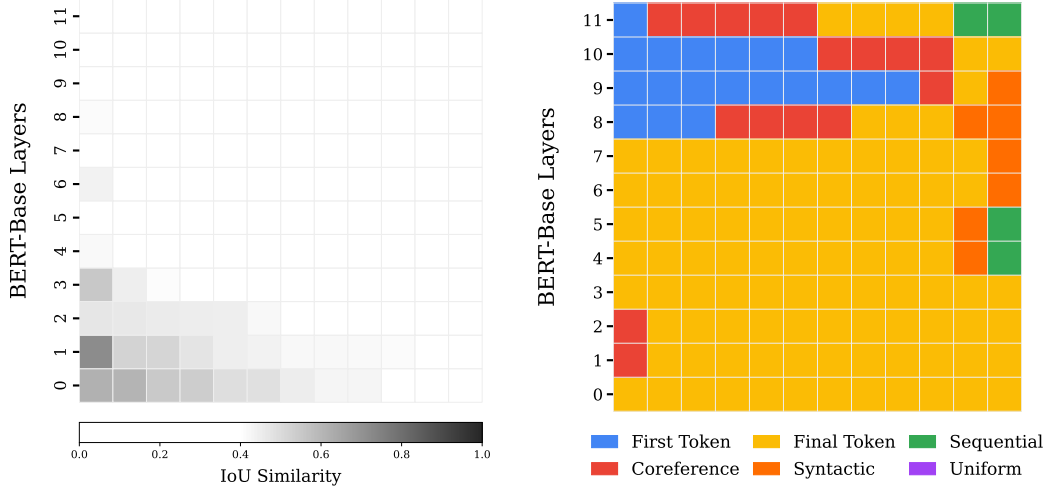
- 368 [34] Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In *International*
369 *Conference on Machine Learning (ICML)*, 2021.
- 370 [35] Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science
371 questions. *arXiv preprint*, 2017. arXiv:1707.06209.
- 372 [36] Kayo Yin and Jacob Steinhardt. Which attention heads matter for in-context learning? *arXiv*
373 *preprint*, February 2025. arXiv:2502.14010.
- 374 [37] Matthew D. Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In
375 *European Conference on Computer Vision (ECCV)*, pages 818–833. Springer, 2014.
- 376 [38] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can
377 a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the*
378 *Association for Computational Linguistics (ACL)*, 2019.
- 379 [39] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. TinyLlama: An open-source
380 small language model. *arXiv preprint*, 2024. arXiv:2401.02385.

A Appendix

A.1 Head to Program Matching

To evaluate the breadth of our synthesized program library Π , we perform a model-wide alignment analysis across all four architectures. For every attention head in each model, we identify the highest-fidelity programmatic proxy within our library and report its maximum alignment score $S = \text{IoU}(A, \hat{A})$.

A.1.1 BERT-base



(a) BERT-base IoU score heatmap. Each cell represents the maximum IoU score achieved by a programmatic proxy for that head.

(b) BERT-base program category heatmap. Circle color indicates the category of the best-fit program for each head.

Figure 7: Head-to-program alignment for BERT-base. Dark cells indicate heads whose behaviors are not yet well-captured by the current program library.

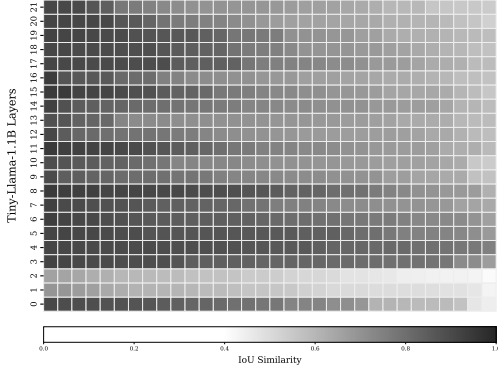
Figure 7 shows that BERT-base is the most poorly characterized model in our analysis, with the majority of heads achieving low single-program IoU scores. High-fidelity matches are sparse and isolated rather than layer-wide, suggesting that the current program library does not yet capture the dominant functional logic of encoder attention. We hypothesize this reflects the inherent difficulty of the masked language modeling objective: compressing bidirectional contextual information into compact attention distributions likely demands more diverse and abstract programs than autoregressive next-token prediction. The scattered high-similarity heads that do exist tend to align with positional and linguistic programs, confirming that these primitives are architecture-agnostic, but the overall coverage gap for BERT represents the clearest direction for future library expansion [6].

A.1.2 TinyLlama-1.1B

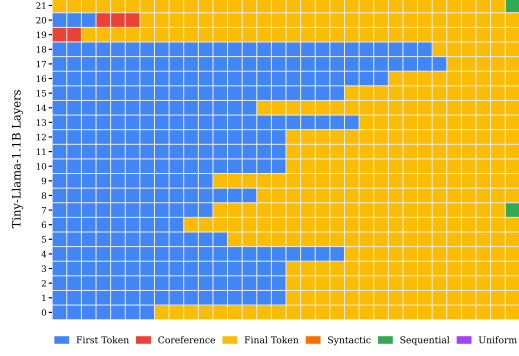
TinyLlama exhibits a pattern broadly similar to GPT-2, with initiation and anchoring programs dominating early layers and more varied program assignments at greater depth. The larger head count per layer (32 heads versus 12 in GPT-2) reveals finer-grained functional specialization, with distinct program categories occupying well-defined layer bands. The near-uniform lower-diagonal structure in many heads reflects the strong positional prior imposed by causal language modeling at this scale.

A.1.3 Llama-3.2-3B

Llama-3B is dominated by a narrow set of recurring programs across nearly all layers and heads, but with generally moderate IoU scores rather than the high-confidence matches seen in GPT-2. This pattern suggests that the current library identifies the correct functional family for most heads but

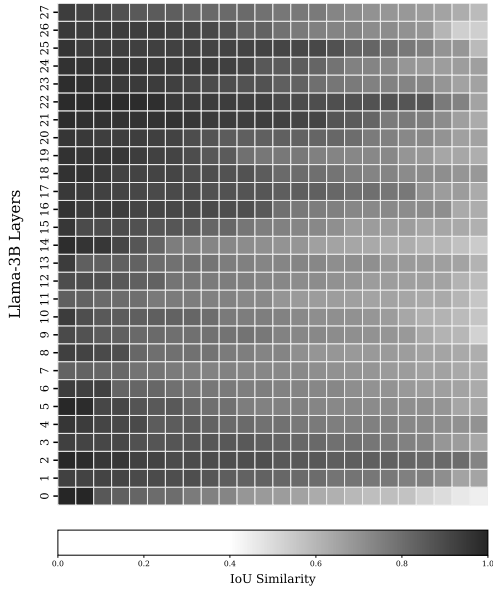


(a) TinyLlama-1.1B IoU score heatmap.

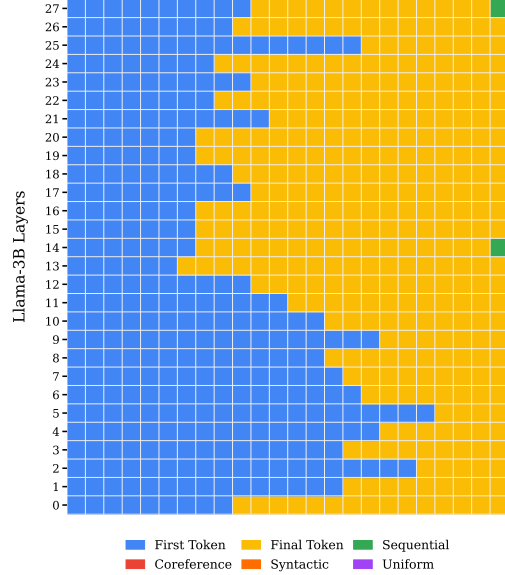


(b) TinyLlama-1.1B program category heatmap.

Figure 8: Head-to-program alignment for TinyLlama-1.1B across 22 layers and 32 heads per layer.



(a) Llama-3.2-3B IoU score heatmap. Dark regions indicate heads whose behaviors are not yet well-captured by the current program library. While the plot shows many heads as being drawn from a small number of clusters, there are substantial differences in behavior within each cluster (e.g. many tokens that primarily attend to the first token, but occasionally implement some other function).



(b) Llama-3.2-3B program category heatmap. Coverage gaps are distributed throughout model depth rather than localized to specific layers.

Figure 9: Head-to-program alignment for Llama-3.2-3B across 28 layers and 24 heads per layer.

lacks the resolution to capture the specific variants operating at this scale. Isolated dark cells appear throughout the depth of the model rather than concentrating in any particular region, indicating that the coverage gap is distributed rather than localized to specific functional stages. Across all four architectures, the same foundational positional and anchoring primitives appear in the earliest layers, supporting the view that basic information-routing behaviors are largely invariant to model scale and that complexity accumulates in how those primitives are extended and combined at greater depth.

413 NeurIPS Paper Checklist

414 1. Claims

415 Answer: [Yes]

416 Justification: The abstract and introduction clearly state the three core contributions: (1)
417 a program synthesis pipeline for approximating attention heads, (2) IoU-based similarity
418 evaluation across four transformer models, and (3) causal validation via head replacement
419 with measured perplexity and downstream task effects.

420 2. Limitations

421 Answer: [Yes]

422 Justification: Limitations are discussed in the Discussion section, including the restriction to
423 attention routing logic (not residual stream features), the empirical 2.5% activation threshold,
424 and the challenge of scaling high-fidelity synthesis to all heads in larger models.

425 3. Theory assumptions and proofs

426 Answer: [N/A]

427 Justification: The paper does not include formal theoretical results or proofs; contributions
428 are empirical.

429 4. Experimental result reproducibility

430 Answer: [Yes]

431 Justification: Section 3 describes the full synthesis pipeline, scoring methodology, datasets,
432 and models.

433 5. Open access to data and code

434 Answer: [Yes]

435 Justification: The program library (1,664 programs across four models) and synthesis
436 pipeline are publicly released. All evaluation datasets (HellaSwag, PIQA, SciQ, ARC-Easy,
437 Social IQA, COPA) are publicly available.

438 6. Experimental setting/details

439 Answer: [Yes]

440 Justification: Section 3.3 specifies all datasets used for activation extraction and downstream
441 evaluation, the four model architectures, activation filtering threshold (top 2.5%), scoring
442 metrics (JSD for synthesis, IoU for evaluation), and the one-step refinement procedure.

443 7. Experiment statistical significance

444 Answer: [No]

445 Justification: Error bars are not reported for IoU similarity scores or downstream task
446 accuracy; results reflect single-run evaluations across 200 sampled questions per benchmark.
447 Reporting variance across runs would require prohibitive recomputation of head replacement
448 across all model-head combinations.

449 8. Experiments compute resources

450 Answer: [Yes]

451 Justification: Section 3.2 reports total API cost for the synthesis phase (approximately \$150,
452 35M input tokens and 3.5M output tokens using Claude Sonnet 4).

453 9. Code of ethics

454 Answer: [Yes]

455 Justification: The research analyzes existing open-weight language models and publicly
456 available datasets. No human subjects, sensitive data, or harmful applications are involved.

457 10. Broader impacts

458 Answer: [Yes]

459 Justification: The paper discusses positive impacts (improved model transparency,
 460 interpretability-grounded editing) in the introduction and conclusion. Potential negative
 461 impacts are limited given the focus on interpretability tooling, but are addressed in the
 462 Discussion section.

463 **11. Safeguards**

464 Answer: [N/A]

465 Justification: The paper does not release new pretrained models or scraped datasets; the
 466 released artifact is a library of symbolic Python programs, which poses no significant misuse
 467 risk.

468 **12. Licenses for existing assets**

469 Answer: [Yes]

470 Justification: All models (BERT, GPT-2, TinyLlama, Llama-3B) and datasets (TinyStories,
 471 tiny-codes, Math500, and all evaluation benchmarks) are cited with original references.
 472 License details are provided in the repository.

473 **13. New assets**

474 Answer: [Yes]

475 Justification: The program library (1,664 programmatic attention head proxies) and synthesis
 476 pipeline are released as the primary artifact of this work, with documentation provided in
 477 the linked repository and described in Section ??.

478 **14. Crowdsourcing and research with human subjects**

479 Answer: [N/A]

480 Justification: The paper involves no crowdsourcing or human subjects research.

481 **15. Institutional review board (IRB) approvals or equivalent for research with human**
 482 **subjects**

483 Answer: [N/A]

484 Justification: No human subjects research was conducted.

485 **16. Declaration of LLM usage**

486 Answer: [Yes]

487 Justification: Claude Sonnet 4 is used as the synthesis agent $\mathcal{S}$ and is a core, non-standard
 488 component of the method. Its role in generating and refining candidate programs is described
 489 in detail in Section 3.2.